Java Backend Learning Sprint

CommaFeed · Maven · Quarkus · JPA · Docker · Keycloak

Мета

За кілька днів тобі потрібно розібратися з основами Java backend development на прикладі реального open-source проєкту CommaFeed.

Це не перевірка того, скільки коду ти можеш згенерувати за допомогою AI. Основна мета — зрозуміти:

- як збирається Java-проєкт;
- як проходить HTTP-запит через backend;
- як працюють REST, JPA, Hibernate і dependency injection;
- як Maven керує залежностями та збіркою;
- як запускати інфраструктуру через Docker;
- що таке Keycloak і як працює service-account авторизація;
- як запускати тести та дебажити код.

Можна використовувати AI, документацію та пошук. Під час фінального інтерв'ю потрібно буде пояснити власний код і виконати невелику зміну без повної передачі задачі AI.

Орієнтовний час: 2–3 дні, до 10–12 годин сумарно.

AI · Як працювати з AI у цьому спринті

Протягом документа після кожного розділу є такі блоки — це підказки, як розібратися швидше за допомогою **Cursor** і **Gemini**.

Розподіл ролей:

- **Cursor** — для роботи з кодом CommaFeed. Для навчання використовуй режим **Ask** (AI лише пояснює і нічого не змінює). Режим **Agent** вмикай тільки тоді, коли можеш своїми словами пояснити, що саме просиш змінити.
- **Gemini** — для понять, помилок і «другої думки»: поясни термін, розбери stack trace, перевір моє пояснення.

Цикл на кожну тему (3 кроки):

1. **Поясни** — попроси пояснити поняття простими словами.
2. **Покажи в коді** — попроси знайти це місце в CommaFeed і відкрий файл сам.
3. **Перевір мене** — напиши своє пояснення і попроси AI знайти в ньому помилки, а не переписати.

GEMINI — СТАРТОВИЙ ПРОМПТ ДЛЯ БУДЬ-ЯКОЇ ТЕМИ

Я вивчаю Java backend. Мій досвід: трохи C++ і базовий Docker. Поясни [тема] простими словами, з однією аналогією з C++ і прикладом коду до 10 рядків. Потім задай мені 3 короткі питання, щоб перевірити, чи я зрозумів. Не давай відповіді, поки я не спробую.

⚠ **Ніколи не вставляй у AI-чат паролі, client secret, access token або вміст .env.**

1. Підготовка репозиторію

1.1. Отримати код

Використовуй CommaFeed версії 7.3.2:

- Repository: https://github.com/Athou/commafeed
- Release: https://github.com/Athou/commafeed/releases/tag/7.3.2

Зроби fork у свій GitHub-акаунт і створи окрему гілку:

```
training/java-backend-fundamentals
```

1.2. Перевірити інструменти

Перевір і запиши версії:

```
java -version
./mvnw --version
docker --version
docker compose version
git --version
```

Додай використані версії до README.

1.3. Початкова перевірка

До внесення змін:

1. Запусти Maven-тести.
2. Збери проєкт.
3. Запусти CommaFeed з H2.
4. Відкрий застосунок у браузері.
5. Створи користувача або виконай початкове налаштування.
6. Увійди в систему.
7. Виклич хоча б один REST endpoint через curl, Postman або браузер.

Якщо щось не працює:

- прочитай текст помилки;
- знайди першу змістовну причину в stack trace;
- запиши проблему і рішення в DEBUGGING.md;
- не видаляй залежності або конфігурацію навмання.

AI · Як швидко розібратися з AI

Аналогія з C++: JDK — це компілятор і стандартна бібліотека разом (як g++ + libstdc++), а JVM — середовище, у якому запускається скомпільований байткод. `java -version` — приблизно те саме, що `g++ --version`.

CURSOR (ASK) — ПЕРШЕ ЗНАЙОМСТВО З РЕПОЗИТОРІЄМ

Я новачок у Java, знаю трохи C++. Поясни структуру папок верхнього рівня цього репозиторію: що лежить у кожній, яка частина — backend, яка — frontend, і з яких 3 файлів мені почати читання. Код не пиши.

> **GEMINI — КОЛИ ЩОСЬ НЕ ЗАПУСКАЄТЬСЯ**
> *Ось помилка з терміналу: [вставити вивід без секретів]. Поясни простими словами, яка перша змістовна причина. Дай 2–3 гіпотези і для кожної — команду, якою я можу її перевірити. Не пропонуй видаляти залежності чи конфігурацію.*

Перевір себе: яку версію Java вимагає CommaFeed 7.3.2 і де в репозиторії це записано?

2. Maven

2.1. Розібрати структуру

Знайди та поясни:

- кореневий pom.xml;
- commafeed-server/pom.xml;
- commafeed-client/pom.xml;
- parent POM;
- Maven modules;
- dependencies;
- plugins;
- properties;
- build profiles.

2.2. Виконати Maven-команди

За допомогою Maven Wrapper виконай і задокументуй команди для:

- перевірки версії Maven;
- очищення build artifacts;
- компіляції;
- запуску тестів;
- запуску тестів лише server-модуля;
- збірки server-модуля разом із залежними модулями;
- створення package;
- запуску CommaFeed у development mode.

Використовуй ./mvnw, а не глобально встановлений mvn.

Приклади команд, які потрібно дослідити:

```
./mvnw clean
./mvnw test
./mvnw clean verify
./mvnw -pl commafeed-server -am test
./mvnw -pl commafeed-server -am package
```

Самостійно знайди правильну команду запуску CommaFeed у development mode та додай її до README.

2.3. Поняття Maven

У LEARNING.md поясни своїми словами:

- що таке Maven;
- що таке pom.xml;
- різницю між dependency та plugin;
- що означають groupId, artifactId і version;
- що таке Maven lifecycle;
- для чого потрібні фази clean, compile, test, package, verify;
- що роблять параметри -pl і -am;
- що таке dependency scope;
- навіщо використовувати Maven Wrapper;
- де Maven зберігає завантажені залежності;
- чим unit tests відрізняються від integration tests.

Для кожного поняття наведи приклад із CommaFeed.

AI · Як швидко розібратися з AI

Аналогія з C++: Maven — це CMake і менеджер пакетів (vcpkg/Conan) в одному. `pom.xml` ≈ `CMakeLists.txt`, dependency ≈ бібліотека, яку лінкуєш, plugin ≈ інструмент, який виконує крок збірки. Завантажені бібліотеки кешуються локально, як кеш пакетів.

CURSOR (ASK) — РОЗІБРАТИ POM.XML

@pom.xml @commafeed-server/pom.xml Поясни для C++-розробника: які тут модулі, які 5 найважливіших залежностей і навіщо кожна, які плагіни і на якій фазі вони працюють. Для кожного пункту вкажи номер рядка, щоб я міг перевірити сам.

GEMINI — ПОЯСНИТИ ВИВІД КОМАНДИ

Ось вивід ./mvnw -pl commafeed-server dependency:tree (скорочено): [вставити]. Поясни, що означають рівні вкладеності і scope (compile, test, provided). Звідки взялася залежність, яку я не бачу в pom.xml?

Практика: запусти одну команду двічі — з -am і без нього — і запиши в LEARNING.md, що змінилося. Власний експеримент запам’ятовується краще за пояснення AI.

Перевір себе: чим відрізняється `./mvnw test` від `./mvnw verify`? Чому `./mvnw` кращий за глобальний `mvn` у команді?

3. Дослідження backend-архітектури

3.1. Простежити існуючий запит

Знайди реалізацію:

```
GET /rest/user/profile
```

Простеж повний шлях запиту:

1. HTTP-запит приходить у застосунок.
2. REST resource приймає запит.
3. Система визначає поточного користувача.
4. Викликається Service або DAO.
5. Дані завантажуються з бази.
6. JPA entity перетворюється на response model або DTO.
7. Клієнту повертається HTTP-відповідь.

Створи REQUEST-FLOW.md.

Для кожного кроку вкажи:

- назву класу;
- назву методу;
- його відповідальність;
- які дані він отримує;
- які дані повертає.

3.2. Поняття Java backend

У LEARNING.md поясни:

- REST;
- HTTP request і response;
- HTTP method;
- status code;
- REST resource;
- controller;
- Service;
- DAO;
- Repository;
- JPA entity;
- DTO;
- Hibernate;
- dependency injection;
- CDI;
- transaction;
- validation;
- exception mapping;
- interface;
- class;
- record;

- enum;
- annotation;
- generics;
- Optional;
- checked і unchecked exceptions;
- immutable object.

3.3. Quarkus і Spring

CommaFeed використовує Quarkus, а не Spring.

Коротко опиши відповідники:

Концепція	Quarkus/Jakarta	Spring
REST endpoint	JAX-RS resource	Spring MVC controller
Dependency injection	CDI	Spring DI
Persistence	JPA/Hibernate	Spring Data JPA/JPA
Transaction	@Transactional	@Transactional
Validation	Jakarta Validation	Spring Validation/Jakarta Validation

Не потрібно переписувати CommaFeed на Spring.

AI · Як швидко розібратися з AI

Аналогії з C++ для мови Java:

- interface ≈ клас лише з чистими віртуальними методами;
- generics ≈ templates, але типи «стираються» після компіляції;
- `Optional` ≈ `std::optional`;
- annotation (`@Path`, `@Inject`) — метадані, які читає фреймворк; найближче в C++ — атрибути на кшталт `[[nodiscard]]`, але значно потужніші;
- пам’ять звільняє garbage collector — `delete` не потрібен;
- checked exceptions у C++ немає: компілятор Java змушує їх обробити або оголосити.

Dependency injection одним реченням: ти не створюєш об’єкти через `new` сам — фреймворк створює їх і передає готовими в конструктор.

CURSOR (ASK) — ПРОСТЕЖИТИ ЗАПИТ

Знайди, де обробляється GET /rest/user/profile. Покажи ланцюжок викликів від анотації @Path до запиту в базу як таблицю: клас → метод → що робить → що повертає. Посилайся на конкретні файли. Код не змінюй.

Після відповіді відкрий кожен файл сам і переконайся, що клас і метод справді існують — AI іноді плутає назви. Найкраща перевірка — breakpoint з розділу 4.

GEMINI — ПЕРЕВІРИТИ ВЛАСНЕ ПОЯСНЕННЯ

Ось як я пояснюю різницю між JPA entity і DTO: [вставити свій текст]. Знайди неточності та скажи, чого бракує. Не переписуй текст за мене.

Перевір себе: чому REST resource не повертає JPA entity напряму? Хто створює об’єкт DAO, якщо в коді немає `new`?

4. Debugging

4.1. Запустити debugger

Запусти commafeed-server у development mode через IDE.

Постав breakpoint у методі, який обробляє:

```
GET /rest/user/profile
```

Виклич endpoint і продемонструй, що debugger зупинився на breakpoint.

4.2. Дослідити виконання

За допомогою debugger:

1. Перевір значення поточного користувача.
2. Подивись локальні змінні.
3. Зроби Step Into у викликаний метод.
4. Зроби Step Over для наступного рядка.
5. Переглянь call stack.
6. Знайди момент створення response object.
7. Перевір, які дані повертаються клієнту.

4.3. DEBUGGING.md

Опиши:

- як запустити debugger;
- де був breakpoint;
- який request ти виконав;
- які класи були в call stack;
- які значення ти переглянув;
- одну проблему, яку debugger допоміг зрозуміти;
- різницю між breakpoint, Step Into, Step Over і Step Out;
- що таке stack trace;
- як знайти root cause помилки у stack trace.

Додай один скриншот debugger із відкритим call stack. Не показуй паролі, токени або інші секрети.

> **AI · Як швидко розібратися з AI**
>
> **Аналогія з C++:** це той самий gdb. Step Into = `step`, Step Over = `next`, Step Out = `finish`, call stack = `bt`.
>
> **Підказка:** Quarkus у dev mode зазвичай відкриває порт для debugger (за замовчуванням 5005), і IDE підключається до нього через «Attach». Знайди це в документації Quarkus, а не лише в поясненні AI.
>
> **CURSOR (ASK) — НАЛАШТУВАТИ DEBUGGER**
> *Я запускаю commafeed-server у Quarkus dev mode. Як підключити debugger у Cursor до запущеного процесу? Які розширення потрібні для Java і як виглядає конфігурація attach у launch.json? Поясни кожне поле.*

GEMINI — РОЗІБРАТИ STACK TRACE

Ось stack trace (без секретів): [вставити]. Познач рядок, який є root cause, і поясни, чому саме він, а не перший рядок. Які рядки належать фреймворку, а які — коду CommaFeed?

Перевір себе: яка різниця між Step Over і Step Out? Чому root cause часто стоїть у блоці «Caused by» внизу stack trace?

5. Практична Java/JPA/REST задача

5.1. Функціональність

Додай можливість зберігати персональні нотатки до feed entries.

Нотатка повинна містити:

- ID;
- користувача-власника;
- feed entry;
- текст;
- дату створення.

Обмеження:

- текст обов’язковий;
- текст після trim() не може бути порожнім;
- максимальна довжина — 500 символів;
- користувач визначається із серверної authenticated session;
- клієнт не повинен передавати user ID у request body;
- користувач може читати лише власні нотатки.

5.2. Створення нотатки

```
POST /rest/entry/{entryId}/notes
Content-Type: application/json

{
  "text": "Read this entry again"
}
```

Очікувані результати:

- 201 Created — нотатку створено;
- 400 Bad Request — текст порожній або задовгий;
- 404 Not Found — feed entry не існує;
- 401 Unauthorized або наявний еквівалент проєкту — користувач не автентифікований.

5.3. Отримання нотаток

```
GET /rest/entry/{entryId}/notes
```

Вимоги:

- повертати тільки нотатки поточного користувача;
- не повертати нотатки інших користувачів;
- сортувати від новіших до старіших;
- не повертати JPA entity напряму;
- використовувати response DTO;
- не включати password, API key або внутрішні поля користувача.

5.4. Архітектура

Реалізація повинна містити:

- JPA entity;
- DAO;
- Service;
- REST resource;
- request DTO;
- response DTO;
- validation;
- перетворення entity у DTO;
- обробку помилок.

Дотримуйся існуючих патернів CommaFeed.

5.5. Тести

Додай тести щонайменше для таких сценаріїв:

1. Успішне створення нотатки.
2. Порожній текст.
3. Текст довший за 500 символів.
4. Feed entry не існує.
5. Нотатки повертаються від новіших до старіших.
6. Користувач бачить власну нотатку.
7. Користувач не бачить нотатку іншого користувача.

Якщо не можеш реалізувати integration test, створи unit test і поясни його обмеження.

AI · Як швидко розібратися з AI

Порядок роботи: спочатку план у режимі Ask → потім пишеш кожен шар сам (entity → DAO → Service → Resource) → після кожного шару збираєш проєкт → AI робить ревю. Режим Agent доречний для чернетки тестів, але кожен тест ти маєш прочитати й пояснити.

CURSOR (ASK) — ЗНАЙТИ ПАТЕРН ДЛЯ КОПІЮВАННЯ

Мені треба додати нотатки до feed entries (вимоги: [вставити 5.1–5.4]). Код не пиши. Знайди в CommaFeed найближчу існуючу функціональність, де є entity + DAO + Service + REST resource, яку варто взяти за зразок. Склади список файлів, які я маю створити або змінити, і поясни роль кожного.

CURSOR (ASK) — ПРО БАЗУ ДАНИХ

Як CommaFeed створює та оновлює таблиці в базі? Якщо я додам нову JPA entity, що ще потрібно, щоб таблиця з'явилася? Покажи існуючий приклад.

CURSOR АБО GEMINI — РЕВЮ БЕЗПЕКИ

Ось мій NoteResource і NoteService: [вставити]. Знайди місця, де користувач може прочитати чужу нотатку або підставити чужий userId. Перевір обробку порожнього тексту, тексту з пробілів і тексту на 501 символ.

Перевір себе: де саме в твоєму коді визначається поточний користувач? Що повернеться, якщо entryId не існує — і який рядок це гарантує?

6. Docker

6.1. Основні поняття

У LEARNING.md поясни:

- Docker image;
- Docker container;
- Dockerfile;
- Docker Compose;
- port mapping;
- environment variable;
- volume;
- network;
- container logs;
- health check;
- різницю між image і container;
- різницю між docker run та docker compose up.

6.2. Практичні команди

Навчися використовувати:

```
docker compose up -d
docker compose ps
docker compose logs
docker compose logs -f keycloak
docker compose stop
docker compose down
```

Поясни різницю між:

```
docker compose stop
docker compose down
docker compose down -v
```

Не використовуй down -v, якщо не розумієш, які дані будуть видалені.

AI · Як швидко розібратися з AI

Твоя перевага: ти вже запускав сервіси в Docker на домашньому сервері. Почни з того, що вже знаєш, і досліджуй лише нове.

Аналогія з C++: image ≈ скомпільований бінарник, container ≈ запущений процес цього бінарника. Один image — багато контейнерів.

> **GEMINI — РОЗІБРАТИ РІЗНИЦЮ НА ПРИКЛАДІ**
> *Поясни різницю між docker compose stop, down і down -v на прикладі контейнера з базою даних: що станеться з контейнером, мережею і даними в кожному випадку? Оформи як таблицю.*

Практика «зламай і полагодь»: навмисно вкажи зайнятий порт або неіснуючий тег image, запусти `docker compose up -d`, знайди помилку в `docker compose logs` і поясни її. Такі експерименти швидко вчать читати логи.

Перевір себе: чому дані зникають після `down -v`, але не після `down`?

7. Keycloak

7.1. Запуск

Створи окремий docker-compose.yml, який запускає Keycloak.

Вимоги:

- використовувати конкретну версію image, не latest;
- admin username і password отримувати з environment variables;
- не комітити реальні секрети;
- додати .env.example;
- додати .env до .gitignore;
- опублікувати Keycloak на локальному порту;
- переконатися через docker compose ps, що контейнер працює;
- перевірити логи контейнера;
- відкрити Keycloak Admin Console.

7.2. Створення realm

Створи realm:

```
commafeed-training
```

У ньому створи:

- realm role directory-viewer;
- звичайного користувача;
- disabled-користувача;
- confidential client;
- service account для confidential client.

7.3. Service account

Створи client для доступу до Keycloak Admin API.

Надай service account лише мінімальні read-only дозволи, необхідні для:

- пошуку користувачів;
- отримання користувача за ID;
- читання ролей;
- читання груп.

Не використовуй повний administrator access, якщо він не потрібен.

7.4. Отримання access token

Через curl отримай access token за допомогою client credentials flow.

Секрет client-а має передаватися через environment variable, а не бути записаним прямо в команду в README.

Після отримання token:

1. Не додавай його до Git.
2. Не вставляй повний token у документацію.
3. Декодуй JWT payload.
4. Знайди claims: iss, sub, aud, exp, client ID, roles.

7.5. Виклик Admin API

Використовуючи access token:

1. Отримай список користувачів realm.
2. Виконай пошук за username або email.
3. Отримай одного користувача за Keycloak UUID.
4. Отримай його realm roles.
5. Отримай його groups.
6. Знайди service-account user.
7. Порівняй звичайного користувача і service-account user.

7.6. KEYCLOAK.md

Поясни своїми словами:

- що таке Keycloak;
- що таке realm;
- що таке client;
- що таке confidential client;
- що таке service account;
- що таке client credentials flow;
- що таке access token;
- що таке JWT;
- що таке claim;
- різницю між authentication та authorization;
- різницю між OIDC login та Admin API;
- що таке Keycloak user UUID;
- чому не варто використовувати username як стабільний ID;

- як визначити service-account user;
- чому client не повинен мати зайвих permissions;
- що станеться після завершення exp токена.

Додай:

- використані curl-запити без секретів;
- приклад скороченої відповіді;
- посилання на офіційну документацію;
- опис щонайменше однієї проблеми, яку довелося продебажити.

Повна інтеграція Keycloak із CommaFeed у межах цієї задачі не потрібна.

AI · Як швидко розібратися з AI

Аналогія з життя: Keycloak — це охорона бізнес-центру. Realm — окрема будівля, client — компанія-орендар, access token — тимчасовий пропуск з терміном дії (exp), roles — двері, які цей пропуск відчиняє.

GEMINI — ЗРОЗУМІТИ FLOW

Поясни OAuth2 client credentials flow як діалог між сервісом і Keycloak, без жаргону. Потім перелічи кроки по порядку: хто що надсилає і що отримує у відповідь.

GEMINI — МІНІМАЛЬНІ ПРАВА

Які мінімальні ролі з клієнта realm-management потрібні service account у Keycloak, щоб лише шукати й читати користувачів, їхні ролі та групи? Для кожної ролі поясни, що вона дозволяє, і дай посилання на розділ офіційної документації.

Відкрий посилання сам — AI іноді вигадує URL або назви ролей. Перевір кожну роль в Admin Console.

CURSOR АБО GEMINI — ДЕКОДУВАТИ ТОКЕН БЕЗПЕЧНО

Дай bash-команду, яка локально декодує payload JWT із змінної середовища $TOKEN і виводить його як форматований JSON. Поясни кожну частину команди.

⚠ Сам токен в AI-чат не вставляй — декодуй його локально.

Перевір себе: чому `sub` у токені service account — не те саме, що client ID? Що станеться, якщо викликати Admin API з простроченим токеном?

8. Робота з AI

AI можна використовувати для:

- пояснення незнайомого коду;
- побудови плану;
- пошуку можливих причин помилки;
- написання чернетки тестів;
- перевірки edge cases;
- пояснення Maven, Docker, JPA та Keycloak.

AI не повинен замінювати власне розуміння.

8.1. DECISIONS.md

Додай мінімум п’ять записів:

```
## Рішення 1

### Проблема
Що потрібно було вирішити?

### Пропозиція AI
Що запропонував AI?

### Перевірка
Як я перевірив пропозицію в коді або офіційній документації?

### Моє рішення
Що я зробив і чому?
```

Хоча б один запис має стосуватися:

- Java/JPA;
- Maven;
- Docker;
- Keycloak;
- тестування.

8.2. Що треба вміти пояснити

На інтерв’ю потрібно буде відповісти:

- яку частину коду написав AI;
- яку частину ти змінив сам;
- яку помилку AI ти знайшов;
- як перевірив правильність рішення;
- що залишилося незрозумілим;
- що вивчив би наступним.

Фраза “так запропонував AI” не є поясненням технічного рішення.

AI · Як швидко розібратися з AI

Веди DECISIONS.md на ходу. Щоразу, коли приймаєш пропозицію AI, одразу запиши 2–3 рядки чернетки. В останній день відновити це з пам’яті майже неможливо.

Як ловити помилки AI:

- попроси посилання на офіційну документацію і відкрий його сам;
- запусти код або команду — компілятор і тести не помиляються так, як AI;
- спитай те саме в Cursor і в Gemini: розбіжність — сигнал перевірити;
- типові помилки AI: неіснуючі методи, застарілі версії бібліотек, Spring-рішення в Quarkus-проєкті.

GEMINI — «АДВОКАТ ДИЯВОЛА»

Ось рішення, яке я прийняв, і чому: [вставити]. Знайди 2 найсильніші аргументи проти. Які ризики я міг пропустити?

9. Git

9.1. Commit history

Не роби один commit із усіма змінами.

Рекомендована структура:

```
docs: describe existing user profile request flow
docs: add Java and Maven learning notes
feat: add entry note entity and dao
feat: add entry note service and REST resource
test: cover entry note behavior
chore: add Keycloak Docker Compose setup
docs: document Docker and Keycloak exercises
docs: record implementation decisions
```

9.2. Перед здачею

Перевір:

```
git status
git diff
git log --oneline
```

У репозиторії не повинно бути:

- .env;
- client secret;
- access token;
- паролів;
- IDE cache;
- build output;
- локальної бази;
- зайвих великих файлів.

AI · Як швидко розібратися з AI

CURSOR (ASK) — ПЕРЕВІРКА ПЕРЕД КОМІТОМ

Ось вивід git status і git diff --stat: [вставити]. Які з цих файлів не варто комітити (секрети, build output, кеш IDE, локальна база)? Що треба додати в .gitignore? Запропонуй розбиття змін на коміти за схемою з розділу 9.1.

Комітити все одно роби сам і перед кожним комітом переглядай `git diff --staged`.

10. Підсумкові матеріали

У pull request або репозиторії мають бути:

- робочий код;
- тести;
- README;
- LEARNING.md;
- REQUEST-FLOW.md;
- DEBUGGING.md;
- KEYCLOAK.md;
- DECISIONS.md;
- docker-compose.yml;
- .env.example;
- зрозуміла історія commit-ів.

README повинен містити:

- вимоги до середовища;
- версії Java, Maven і Docker;
- команди збірки;
- команди запуску;
- команди тестування;
- запуск Keycloak;
- приклади curl для нотаток;
- приклади curl для Keycloak без секретів;
- опис незавершеної роботи.

11. Що потрібно продемонструвати на інтерв’ю

Підготуй 10–15-хвилинну демонстрацію:

1. Покажи структуру Maven-проєкту.
2. Збери commafeed-server.
3. Запусти тести.
4. Запусти CommaFeed.
5. Постав breakpoint.
6. Виклич endpoint і покажи call stack.
7. Створи нотатку через curl.
8. Отримай нотатки поточного користувача.
9. Покажи, що інший користувач не бачить чужі нотатки.
10. Запусти Keycloak через Docker Compose.
11. Покажи стан і логи контейнера.
12. Отримай access token.
13. Виклич Keycloak Admin API.
14. Покажи звичайного та service-account користувача.
15. Розкажи про одну помилку AI, яку ти знайшов або виправив.

AI · Як підготуватися з AI

Порада: прожени демонстрацію повністю хоча б один раз із таймером і з нуля (після `docker compose down`). Більшість проблем на демо — це забута команда або незапущений контейнер.

GEMINI — РЕПЕТИЦІЯ ІНТЕРВ’Ю

Ти технічний інтерв’юер. Я реалізував нотатки до feed entries у CommaFeed (Java, Quarkus, JPA) і налаштував Keycloak service account. Ось мої LEARNING.md і DECISIONS.md: [вставити]. Задавай мені по одному питанню «чому?» і «що буде, якщо…?». Після кожної відповіді оцінюй її та кажи, чого бракувало.

12. Live-завдання на інтерв’ю

Після демонстрації потрібно буде виконати невелику зміну:

```
DELETE /rest/entry/{entryId}/notes/{noteId}
```

Вимоги:

- видалити нотатку може лише її власник;
- не можна передавати user ID від клієнта;
- для чужої або відсутньої нотатки не потрібно розкривати, чи вона існує;
- операція повинна бути транзакційною;
- потрібно додати хоча б один тест;
- потрібно пояснювати свої дії під час роботи.

На live-завдання буде приблизно 20–30 хвилин.

AI · Як підготуватися з AI

Live-завдання перевіряє, наскільки добре ти знаєш власний код. Найкраща підготовка — мати змогу без AI відповісти: де визначається поточний користувач, як DAO шукає нотатки, де стоїть `@Transactional` і як влаштовані твої тести.

GEMINI — ПОТРЕНУВАТИ ПОЯСНЕННЯ ВГОЛОС

Я пояснюю свій код вголос під час live-кодингу. Ось мій NoteService: [вставити]. Опиши, як досвідчений розробник пояснював би цей код рядок за рядком за 2 хвилини, а потім попроси мене пояснити його своїми словами.

13. Критерії успіху

Ідеальна реалізація не є обов’язковою.

Найважливіше:

- ти самостійно запустив проєкт;
- можеш пояснити шлях HTTP-запиту;
- розумієш відповідальність REST resource, Service і DAO;
- розумієш базові JPA entities та relations;
- розумієш роль транзакції;
- можеш використовувати Maven для збірки й тестування;
- можеш запускати та діагностувати Docker-контейнери;
- розумієш базові поняття Keycloak;
- можеш дебажити код через breakpoint і call stack;
- пишеш тести;
- не комітиш секрети;
- перевіряєш пропозиції AI;
- чесно пояснюєш, чого ще не знаєш.

Якщо щось не встиг — опиши:

1. що саме не завершено;
2. чому;
3. що вже перевірено;
4. який наступний крок;
5. скільки приблизно часу потрібно.